
📄 PDF RAG Chatbot - Technical Report

Fine-Tuned Retrieval-Augmented Generation System

---

1. Document Structure and Chunking Logic

Document Processing Pipeline

The system processes PDF documents through a structured pipeline to enable retrieval-based question answering.

Step 1: Text Extraction

PDF text is extracted using `PyPDFLoader` from `langchain_community`. The extraction preserves structural metadata and ensures clean raw text input.

Step 2: Text Cleaning

A preprocessing module in `pipeline.py` performs normalization using regex operations:

- Removal of excessive whitespace
- Removal of URLs (`http`, `https`, `www`)
- Stripping of unnecessary leading/trailing spaces

This ensures consistent input for downstream embedding generation.

Step 3: Chunking Strategy

Documents are split using `RecursiveCharacterTextSplitter` with:

- Chunk Size: 1000 characters

- Chunk Overlap: 200 characters

The splitter prioritizes semantic boundaries (paragraphs → sentences → words), ensuring meaning is preserved across chunks.

Rationale:

- Improves retrieval precision
- Maintains contextual continuity
- Prevents loss of information at chunk boundaries

Output Artifacts

- `chunks/chunks.txt` → raw chunked text
- `chunks/chunks.json` → structured metadata (page number, chunk index, source)
- `vectordb/index.faiss` → FAISS vector index

---

2. Embedding Model and Vector Database

Embedding Model

BAAI/bge-small-en-v1.5

This model is used via `SentenceTransformer` to generate dense vector embeddings.

Justification:

- Strong semantic similarity performance
- Lightweight and efficient for local execution
- Suitable for retrieval-based QA systems

Each chunk is converted into a **384-dimensional embedding vector**.

Vector Database (FAISS)

FAISS is used for fast similarity search.

- Vectors are L2-normalized
- Inner Product (IndexFlatIP) is used for cosine similarity

- Top-K retrieval (K=5) is applied during query time

Why FAISS:

- High-speed nearest neighbor search
- Efficient memory usage
- Scalable for document-level retrieval

Retrieval Process

1. User query is embedded using same model
2. Query vector is normalized
3. FAISS retrieves top-K relevant chunks
4. Irrelevant results are filtered using similarity threshold

---

3. Prompt Format and Generation Logic

Prompt Template (Strict Grounding)

```python
You are a document QA assistant.

Use ONLY the supplied context.

Rules:

1. Use ONLY the supplied context.
2. If the question asks for a specific number of items (e.g., "five things", "three rights"),
    return EXACTLY that many.
3. If the context contains more items than requested, return only the first N items
    (N = number in question).
4. If the context contains fewer items than requested, return all available items and note the
    shortage.
```

```
5. Do NOT add extra text, repeat the question, or include unrelated
content.
If information is absent, respond: NOT FOUND IN DOCUMENT


CONTEXT:
{context}

QUESTION:
{query}

ANSWER:
```

Generation Logic

- Retrieved context is injected into prompt
- LLaMA 3.3 (70B Instruct via Groq API) is used
- Temperature = 1
- Streaming enabled (`stream=True`)
- Token-wise output rendered in Streamlit UI

---

4. Example Queries and Observed RAG Performance

The following results are based on evaluation of real system outputs using a legal/contract-style eBay document.

Success Case 1: Complaint Information

Query:
Where can California residents report complaints?

Response:
400 R Street, Sacramento, CA 95814, or by telephone at (800) 952-5210.

Observation:
Correct retrieval of structured contact information with high factual accuracy.

Success Case 2: Implied Warranties

Query:
What are the three implied warranties that eBay expressly excludes?

Response:

- Implied warranties of merchantability
- Fitness for a particular purpose
- Non-infringement

Observation:
Accurate extraction of legal enumerations with correct structure preservation.

Success Case 3: Arbitration Exceptions

Query:
What are the two exceptions to the Agreement to Arbitrate?

Response:

1. Small claims court eligibility
2. Equitable relief for intellectual property infringement

Observation:
System correctly identifies conditional legal exceptions and maintains structured output.

Success Case 4: Vehicle Listing Restrictions

Query:
What are the five things eBay has never done with respect to vehicle listings?

Response:

1. Held title for any vehicle
2. Inspected any vehicle
3. Set or negotiated prices

4. Possessed any vehicle
5. Shipped or delivered any vehicle

Observation:
Correct enforcement of “exact number of items” constraint (returns 5 items accurately).

Failure Case (Controlled Abstention)

Query:
Who owns eBay?

Response:
NOT FOUND IN DOCUMENT

Observation:
The system correctly avoids hallucination when information is absent from retrieved context.

---

5. Limitations and Performance Considerations

Hallucination Control

The system is designed with strict grounding constraints:

- No external knowledge usage
- Forced abstention when context is missing
- Strong reduction of hallucinations

System Limitations

- Single PDF processing (no multi-document support)
- Fixed chunk size may not adapt to all document types
- No query expansion or reranking
- Retrieval quality depends on embedding coverage
- No persistent conversational memory

Performance Constraints

- Embedding generation time increases linearly with document size
- CPU-based execution may cause latency for large PDFs
- Initial model loading introduces startup delay

---

6. Conclusion

This project implements a complete Retrieval-Augmented Generation (RAG) pipeline combining semantic search with large language model reasoning.

By integrating FAISS-based retrieval, BAAI/bge-small-en-v1.5 embeddings, and LLaMA 3.3 (Groq API), the system achieves accurate, context-grounded responses with strong hallucination control.

Evaluation results demonstrate:

- High retrieval accuracy
- Correct structured extraction (lists and constraints)
- Reliable abstention on missing knowledge queries

Overall, the system is suitable for document-based question answering tasks requiring factual accuracy and controlled generation.

Report Date: June 1, 2026
System Components: FAISS, BGE-small-en-v1.5, LLaMA 3.3 (Groq), Streamlit
System Type: Retrieval-Augmented Generation (RAG)
